

Recall-Bounded Durability for Graph-Based Approximate Nearest Neighbor Indexes

Anonymous
Under submission

July 19, 2026

Two-host durability study; committer validation on Apple silicon

Abstract

Approximate nearest-neighbor (ANN) work usually reports recall and queries per second but not whether an acknowledged insert survives failure. We make durability a measured axis and implement an opt-in *recall-bounded committer* in a segmented C++20 HNSW store. The shipped default remains durable-before-return. In weak mode, one leader serializes WAL appends and admission, exposes complete records through an exact volatile tail, and hardens that tail with versioned WAL fences. Monotone appended, visible, and durable LSNs, reserved LSN high-water ranges, bounded WAL parsing, and a two-phase seal make production recovery accept only complete fenced generations. Volatile records never mutate a queried HNSW, so a durability fence changes classification but not query representation.

The mechanism supports a distribution-free strict cap on weak-tail size, $W \leq \lfloor \varepsilon k_{\min} \rfloor$, for minimum weak-visible query width $k_{\min}$; latest queries below that width are served stable-only. It also supports exchangeable-mean and hypergeometric policies, and an online CUSUM guard that fences and falls back to stable ACKs when fresh records are query-hot. We validate it with 15/15 real process-crash frontiers using `_exit`, 661/661 WAL byte-prefix images, and 2,880 query observations reopened through production read-only recovery. Every trial has zero stable-ACK loss, no unexpected weak survivor, and no admission-cap overshoot; all query observations have zero topology amplification (within 10^{-12}). Strict observations satisfy $\Delta^+ \leq L = M \leq 0.1 \leq \varepsilon = 0.2$, with observed maxima $\max \Delta^+ = \max L = \max M = 0.1$.

On the canonical Apple-silicon `F_FULLFSYNC` run (four writers, 30 image executions per case), stable group commit has median 299 writes/s (227–342), while exchangeable random has median 475 (317–539); their observed ranges overlap. Strict-random has median 196 writes/s (151–238), and the hot guard has median 117 (81–176). A pre-committer two-host study separately shows the tax, group-commit curve, and snapshot/WAL recovery gap on Apple and AWS Linux/NVMe. The committer evaluation is insert-only, single-node, and unfiltered; its Linux `dm-log-writes` harness is implemented, but block-replay execution remains pending suitable disposable hardware.

1 Introduction

Graph-based ANN indexes—Hierarchical Navigable Small World (HNSW) graphs [1] and their descendants—dominate in-memory vector search, and vector databases built on them now back retrieval-augmented generation and semantic search at scale. The community evaluates them on *recall@k* (the fraction of a query’s true k nearest neighbors the index returns) and *throughput* (queries/s, QPS), summarized by the recall-vs-QPS Pareto frontier popularized by ANN-Benchmarks [13] and the Big ANN challenges [14]. What these evaluations almost never measure is *durability*: does an acknowledged write survive a crash?

Vector databases now ingest continuously, yet mainstream ANN libraries omit durable storage (Faiss offers manual serialization [10]; hnswlib has no crash contract [2]), and common benchmarks omit crash recovery and `fsync` [13, 14, 15]. Production systems such as Milvus [12] use WALs but publish no isolated durability analysis. Research systems either inherit a host DB’s durability [9] or, like LSM-VEC [5], report no synchronization or recovery cost.

We use known storage techniques in a standalone graph-ANN index and measure where naive evaluation misleads; group commit and snapshots are not claimed as novel. Our ANN-specific WAL path bounds a crash’s top- k exposure while isolating volatile records from the graph (§6). Legacy measurements span Apple and AWS Linux; the committer experiment uses one Apple host.

Contributions.

- **C1 (§3).** An ANN durability protocol with tmpfs detection, a sync-floor baseline, and explicit `fsync`-mode labeling.
- **C2 (§4).** An illustrative pre-committer mutable-HNSW durability-tax measurement versus durability strength, with an AWS Linux cross-check.
- **C3 (§5).** A pre-committer mutable-HNSW group-commit *batch-size sweep* showing throughput versus batch size and `fsync` mode.
- **C4 (§7).** A pre-committer baseline comparing snapshot load with graph-rebuilding WAL replay on block storage, complementing P-HNSW’s persistent-memory analysis [11].
- **C5 (§6).** *Recall-bounded durability*: we bound top- k exposure in an opt-in committer with fenced WAL generations, isolated raw deltas, three admission policies, and correlation-triggered stable fallback, validated by real crashes and exhaustive WAL prefixes.

2 Background and System

Our testbed is a from-scratch C++20 vector database with a segmented store. Its query representation has three layers: immutable sealed HNSW snapshots; a durable raw delta R ; and, only when opt-in weak acknowledgment is enabled, a volatile raw tail U . Latest reads search all three layers, stable reads omit U , and the results are merged by the deterministic key (distance, key). Both raw layers use exact distance scans. In particular, inserting into U and later hardening it into R never mutates an HNSW. The existing insert API remains stable-by-default and returns only after synchronization. When recall commit is enabled, every latest-search API—scalar, metadata, and batch—serves $k < k_{\min}$ from the stable snapshot and reports the effective visibility on the structured response. Thus every query that can observe U lies inside the admission contract.

Durability model. Version-2 WAL mutation frames contain explicit lengths, an LSN, operation, payload, and a header-plus-payload CRC. A fence covers a generation by recording its LSN range, mutation count, rolling hash, and its own CRC. The sole WAL appender maintains appended $\geq$ visible $\geq$ durable; frame append precedes in-memory publication internally, while current public status snapshots report appended = visible $\geq$ durable. It reserves LSN ranges in a CRC-protected high-water file before assignment, so discarded weak-tail LSNs are never reused. Recovery bounds lengths before allocation, validates mutations and fences, and accepts only complete fenced generations. Writable recovery truncates and synchronizes any suffix; the read-only verifier follows the same parse without modifying the image. Legacy version-1 WALs are treated as stable

and upgraded before weak mode can be enabled. In strong macOS mode, a failed `F_FULLFSYNC` is retried only for interruption and otherwise propagates; it never downgrades to plain `fsync`, and a failed fence cannot return a stable ACK.

Sealing is likewise crash-safe: it writes and synchronizes a snapshot plus `seal.ready`, creates the replacement mutable generation, atomically publishes the manifest, and only then retires the old WAL. Failpoints cover each publication frontier. Recovery of a sealed segment loads its snapshot rather than rebuilding the graph from its WAL.

Index. Search uses HNSW with the Malkov–Yashunin diversifying neighbor-selection heuristic [1]. The search-time candidate-list width `ef` (and its build-time analogue `efconstruction`) trade recall for speed. Distance kernels are SIMD-vectorized (NEON on our host; an AVX+FMA path is implemented but untested here) with a devirtualized hot path. Recall@10 on SIFT1M rises with `ef` to 0.999 and is on the same recall-QPS frontier as `hnswlib` (Table 6), establishing a competitive index before durability is added.

3 Measuring Durability Honestly

Durability numbers are worthless without a statement of *what* was made durable. The pitfalls below are well known in the storage-systems literature— application-level crash-consistency is hard to get right [21], commodity stores mishandle power faults [22], and even PostgreSQL mishandled `fsync` error reporting [23]. Our contribution is not to discover them but to port this discipline to ANN evaluation, where it is absent. Two traps recur.

The `tmpfs` trap. Benchmarks frequently run under `/tmp`, which on some Linux distributions is a RAM-backed `tmpfs`; synchronization there cannot demonstrate persistence across loss of power. Such an insert benchmark measures the memory-backed path only. Our harness `statfs`-checks the target and *refuses* to report write numbers on `tmpfs`.

The `fsync-mode` trap. Apple documents that `fsync(2)` does not necessarily flush a drive’s volatile cache; `fcntl(F_FULLFSYNC)` asks the device to complete that stronger operation. A number labeled “power-loss durable” under plain macOS `fsync` therefore overstates its contract. Our *sync floor*, a tight `write()+sync` loop, separates device synchronization from index work. The $155\times$ mode gap in Table 1 is a macOS/APFS device microbenchmark, not a portable system speedup; the transferable result is the contract distinction.

Methodology and statistics. The legacy durability study reports the median and observed range of seven trials after warm-up; larger- N sweeps and the extended recovery experiment use three. Seven observations do not justify a parametric confidence interval, so differences smaller than overlapping ranges are treated as noise. The committer study is separate: it uses 30 independently executed tail/write images per case and shuffles case order within each repetition. Its unit of analysis is an image: recovery queries are averaged within an image, then min/median/max are reported across images. We do not bootstrap correlated query rows. All images share one base graph, query set, HNSW seed, and crash frontier, so they are execution repetitions rather than independent workload or graph samples. SIFT1M recall is one fixed build per index, with single-run query timing only, so we do not call it deterministic across independently randomized graph builds.

Table 1: Sync-call floor in a tight `write()+sync` loop, median of 7 trials (min–max). Plain macOS/APFS `fsync` does not promise to force a volatile device cache, whereas `F_FULLFSYNC` requests that stronger operation; Linux `fsync` has the stronger storage-completion contract. The legacy loop did not record syscall return values, so these are timing measurements, not evidence of power-loss survival. The 155 $\times$ ratio is the same-host macOS contract gap.

Platform	Requested mode	Loop calls/s (median, min–max)
macOS/APFS	Plain <code>fsync</code> (weaker contract)	49,199 (41,983–51,782)
macOS/APFS	<code>F_FULLFSYNC</code> (stronger contract)	317 (291–324)
Linux/NVMe	Plain <code>fsync</code> (Linux contract)	527 (511–529)

Table 2: Pre-committer mutable-HNSW durability tax: single-insert throughput and latency vs. `fsync` mode (Apple M4 Pro, $d=128$, per-write durability, $N=2,000$ inserts/trial), median of 7 trials (min–max). Honest durability cuts throughput to $\sim 22\%$ ($4.5\times$) and roughly triples the p99 (99th-percentile) tail.

Mode	Inserts/s	p50 (μ s)	p99 (μ s)
Plain <code>fsync</code>	442 (408–476)	1,732 (1,435–1,896)	14,425 (14,114–14,915)
<code>F_FULLFSYNC</code>	98 (98–106)	8,097 (7,972–8,226)	48,372 (35,252–49,390)

Cross-platform validation (Linux/NVMe). To confirm the phenomena are not macOS artifacts, we ran the legacy suite on an AWS Linux host (x86-64, ext4 on NVMe), where `fsync` requests the storage-completion contract used by the application. The stronger-contract per-write rate is 378 inserts/s (vs. the 527/s floor), group commit yields $4.9\times$ (Table 3 shows $4.3\times$ on macOS `F_FULLFSYNC`), and the recovery gap reaches $128\times$ at $N=6k$ with sealing cost (29–47 ms) again far below the WAL replay it saves (385–4843 ms). So the durability tax, the group-commit curve, and the snapshot-vs-replay recovery result all hold on the platform vector databases actually run on—the macOS numbers are illustrative, but these legacy tax and recovery trends are not confined to it. The new committer measurements and process-crash matrix, however, were run only on the Apple host; its physical block-replay experiment remains pending (§10).

4 Pre-Committer Durability Tax

We define the *durability tax* as the throughput an index loses and the latency it adds to guarantee that acknowledged writes survive. We measure it by inserting one record at a time (each `fsync`’d before returning) into the archived pre-committer segmented store, where each mutable insert also mutated HNSW, and vary *only* the `fsync` mode. The SIFT-scale configuration uses $d=128$ and $ef_{\text{construction}}=200$. Because the two rows in Table 2 run the identical index work and differ only in whether the per-insert flush actually reaches the drive, the difference between them isolates the durability cost for that engine. Since commit 4017fa7, the current engine exact-scans a raw mutable delta and builds HNSW only at seal; current `make bench-durability` therefore does not reproduce these rows.

Stronger durability cuts throughput to about a fifth and roughly triples p99. We do not subtract the single-sync floor: an insert performs multiple durable operations, so that floor is a lower bound rather than a clean latency component. Under plain `fsync`, graph insertion dominates; this both understates the stronger contract’s cost and masks batching’s benefit (§5).

Table 3: Pre-committer mutable-HNSW group-commit batch-size sweep (Apple M4 Pro, $d=128$, $N=2,000$ inserts/trial, one `fsync` per batch), inserts/s as median of 7 trials (min–max), and speedup over batch 1. Under `F_FULLFSYNC` the speedup climbs a curve to $4.3\times$; under plain `fsync` batching barely helps because synchronization was already cheap in this measurement. Speedups are computed on unrounded medians.

Batch	F_FULLFSYNC (honest)		Plain <code>fsync</code>	
	Inserts/s	Speedup	Inserts/s	Speedup
1	102 (97–109)	1.0 $\times$	481 (415–491)	1.0 $\times$
10	339 (318–356)	3.3 $\times$	498 (453–533)	1.0 $\times$
50	402 (391–443)	4.0 $\times$	509 (462–560)	1.1 $\times$
200	419 (410–456)	4.1 $\times$	496 (443–563)	1.0 $\times$
1000	431 (411–461)	4.2 $\times$	514 (440–539)	1.1 $\times$
2000	433 (418–461)	4.3 $\times$	507 (447–543)	1.1 $\times$

Scale checks. In a three-trial size sweep, the plain-to-`F_FULLFSYNC` throughput ratio falls from $4.5\times$ at $N = 2k$ to $2.4\times$ at 10k and $2.1\times$ at 30k as graph work grows relative to fixed synchronization cost; group-commit benefit falls similarly. The small trial count and wide raw ranges make this a trend, not a fitted scaling law.

5 Pre-Committer Group Commit for a Graph-ANN WAL

Group commit is a classic database technique [16]: batch several log records and issue a single `fsync` for the group, decoupling per-operation commit latency from flush latency. In the archived mutable-HNSW engine, we added a deferred-sync mode to the mutable segment: during a batch, WAL appends skip the per-record `fsync`, and a single `commitDeferredSync()` at the end `fsyncs` once for the whole batch. The then-current public batch-insert path used it; single inserts kept per-write durability. As in §4, current raw-delta code has different index work.

Under `F_FULLFSYNC`, throughput rises from 102 to 433 inserts/s ($4.3\times$) as one synchronization covers more inserts; plain-`fsync` throughput is essentially flat. Batch 1 and Table 2 are independent runs, so their ranges carry the signal. Acknowledgments remain stable and follow the shared synchronization.

Crash scope. The stable path is exercised inside the committer’s 15-frontier `_exit` matrix (§6); the child exits while its database object is live, and the parent reopens that database through production recovery. This tests process death and the OS-visible disk image, not loss of device caches. We implemented a separate Linux `dm-log-writes` harness that records an external acknowledgment ledger and replays each block-log entry before read-only verification. That physical committer-path run has *not* yet been executed on disposable hardware, so it contributes no result here.

6 Recall-Bounded Durability

Standard group commit does not weaken durability: it releases a group only after the shared synchronization. Our separate, opt-in weak-ACK API may return after a complete WAL mutation becomes visible but before a fence is synchronized. Let D be the total durable population across

sealed HNSWs and raw R , let $W = |U|$, and let $k_{\min}$ be the smallest query width allowed to observe U . Latest requests with $k < k_{\min}$ are served from the stable snapshot. For a query with frozen exact pre-crash top- k set T , define $M = |T \cap U|/k$.

Admission contracts. Strict admission requires

$$W + m \leq \lfloor \varepsilon k_{\min} \rfloor$$

for an atomic request of m records. It is distribution-free; importantly, the cap is zero when $\varepsilon < 1/k_{\min}$, so such a configuration transparently uses stable group commit. Exchangeable-mean admission instead requires $(W + m)/(D + W + m) \leq \varepsilon$. If insertion order is independent of a query, $|T \cap U|$ is hypergeometric and $\mathbb{E}[M] = W/(D + W)$. This is a population-mean model, never a per-query guarantee. A third policy admits only when the exact hypergeometric tail

$$\Pr[\text{Hypergeom}(D + W + m, W + m, k_{\min}) > \lfloor \varepsilon k_{\min} \rfloor] \leq \delta.$$

The implementation also enforces record, byte, and age limits. If a request does not fit, it fences the current tail and retries; an indivisible request that still does not fit receives a stable ACK. No policy silently forces $W \geq 1$. Therefore every servable query that includes weak records has $k \geq k_{\min}$, closing the otherwise-invalid small- k case.

Commit and query path. Followers enqueue writes to one leader, which is the sole WAL appender and serializes admission; concurrent writers therefore cannot overshoot the cap. A weak insert is released only after its complete frame and U entry are visible. A stable request, explicit fence, age/size limit, seal, or shutdown closes the generation; after successful stable synchronization, the leader atomically advances the durable frontier and reclassifies U into R . The group-delay wait is clipped to the oldest weak record’s age deadline, so the worker begins the age fence before processing queued work; device synchronization latency remains additive. In requested `F_FULLFSYNC` mode, failure is surfaced, telemetry records no strong-sync success, and the committer enters a failed state rather than returning `Stable` after plain `fsync`. The vectors remain in the same exact representation. The marginal weak-tail scan cost is $\Theta(Wd)$, and the total mutable exact scan is $\Theta((|R| + W)d)$, so weak admission exchanges synchronization work for query work; it is not free ANN capacity.

Exchangeability is monitored rather than assumed silently. Every latest query entry point, including segmented batch search, reports volatile hits to a one-sided CUSUM against the expected hit rate $W/(D + W)$. An alarm blocks further exchangeable weak admissions, fences the tail, and serves subsequent weak requests with stable ACKs. This is a drift detector, not a proof and not retroactive protection; only the configured strict policy is an adversarial weak-ACK guarantee.

Isolation invariant. Let $S \subseteq U$ be any records from the pre-crash volatile tail that are present after recovery, and let the stable candidate set and deterministic merge be identical before and after recovery. Because U never mutates an HNSW, the exact membership loss is

$$L = \frac{|T \cap (U - S)|}{k}, \quad \Delta^+ = \max(0, \text{recall}_{pre} - \text{recall}_{rec}) \leq L \leq M.$$

We define topology amplification as $\max(0, \Delta^+ - L)$. Under isolation, loss is intentionally independent of HNSW ef: the durable topology and candidate fingerprint must remain unchanged. The evidence is therefore zero amplification plus a fingerprint check, not a fabricated ef effect.

Table 4: Canonical Apple-silicon F_FULLFSYNC run, 30 images/case. Performance is median [min,max]; ACK p50 is weak (W) or stable (S); syncs cover the timed writer interval. Loss is overall mean; image min/median/max, without a query-row bootstrap. Fixed-count/time controls are omitted here but remain in all invariant totals and raw CSVs.

Policy/workload	$W_{\max}$	writes/s	ACK p50 (ms)	Syncs	mean; image min/med/max	max
Stable group	0	299 [227,342]	S 11.8 [10.8,14.5]	7	0; 0/0/0	0
Strict/random	2	196 [151,238]	W 18.1 [16.0,23.4]	12	.007; .000/.008/.017	.10
Strict/hot	2	222 [141,268]	W 16.9 [14.9,31.7]	12	.025; .025/.025/.025	.10
Exchange/random	9	475 [317,539]	W 6.4 [5.8,12.7]	2	.051; .017/.050/.083	.30
Exchange/hot	9	473 [247,547]	W 6.5 [5.6,17.1]	2	.225; .225/.225/.225	.30
Exchange+guard/hot	4	117 [81,176]	W 3.6 [2.3,5.2]	22	0; 0/0/0	0

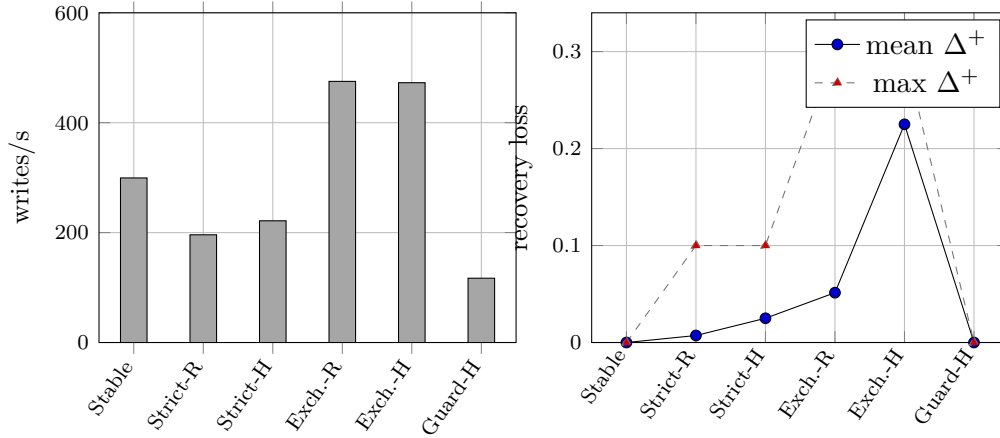


Figure 1: Median committer throughput and recovery loss from the same canonical run (min/max are in Table 4). R/H denote random/hot. Exchangeable mode has a higher median but overlapping range and is model-conditional; the hot guard alarms, fences, and falls back.

Validation protocol. Three tests exercise different failure models. First, the child calls `_exit` with a live database at 15 predeclared admission, fence, and seal frontiers; all 15 read-only verification, writable reopen, and re-verification cases pass. Second, an exhaustive fixture opens all 661 byte-length prefixes of a 660-byte WAL through the production read-only recovery path; each exposes exactly the last complete fence and never its weak tail. These are byte-prefix images, not physical torn-write experiments. Third, the end-to-end benchmark clones live disk images and re-opens them through the same production path. It uses 160 clustered 12-dimensional base vectors, 25 writes from four concurrent writers, $k = 10$, 30 image executions, and 12 recovery queries for each of eight cases: 240 cloned live images and 2,880 recovery-query observations in total. Case order is shuffled within each repetition. The data are deliberately small enough to audit; they are not SIFT1M or a scale result. Hypergeometric admission is unit-tested but has no performance row in this benchmark.

Results. Table 4 and Fig. 1 summarize the canonical run. Every aggregate row has zero stable-ACK loss, zero unexpected weak survivor, and zero cap overshoot. All 2,880 recovery-query rows satisfy $\text{LostIDs} \subseteq U_{pre}$. Amplification is zero within 10^{-12} (the only nonzero machine value is 5.55×10^{-17}). In strict mode with $\varepsilon = 0.2$ and $k = 10$, every observation satisfies $\Delta^+ \leq L =$

Table 5: Cold-open recovery time, sealed (snapshot load) vs. mutable (WAL replay), Apple M4 Pro, $d=128$, `F_FULLFSYNC`; median of 7 trials (min–max). Snapshot-creation/sealing cost is *not* included—only the cost of reading an already-sealed snapshot. WAL-replay time is dominated by graph rebuild and is `fsync`-mode-independent.

	N	Sealed (ms)	WAL replay (ms)	Gap
	1,000	38.9 (32.4–46.8)	195.5 (181.7–196.5)	$5.0\times$
	3,000	44.2 (33.8–55.0)	650.7 (647.0–683.9)	$14.7\times$
	6,000	47.0 (42.7–48.4)	1,483.6 (1,455.6–1,602.0)	$31.6\times$

$M \leq 0.1 \leq 0.2$, and the observed maxima of all three terms are 0.1; equality $L = M$ holds here because no weak record survives recovery. A separate reversed-order base built with seed 101 is rejected against the seed-100 configuration on persisted-seed metadata (with no observed answer drift); normal runs require equal stable-answer fingerprints.

The performance tradeoff is large and noisy. Stable group commit’s median is 299 writes/s; exchangeable-random’s is 475, but their observed ranges (227–342 and 317–539) overlap. Exchangeable-hot spans 247–547, so the run does not establish a strict exchangeable-over-stable ordering. Timed synchronization counts are 7 for stable, 12 for strict, and 2 for unguarded exchangeable rows. For target $\varepsilon = .05$, random image-mean loss has mean .051, median .050, and range .017–.083; hot loss is .225 throughout. Exchangeability is therefore a workload model, not a per-query promise.

Strict mode preserves the distribution-free bound across the full throughput ranges in Table 4. The hot guard alarms in all 30 images and has median 117 writes/s while fencing and serving later writes stably; its weak p50 is 3.6 ms and stable-fallback p50 is 36.7 ms. Median query p95 is 6.8/7.0 ms for unguarded exchangeable random/hot and 29.0 ms with the guard. These are one-host ranges, not deployment claims. Exact-tail scaling is not isolated; only its $\Theta(Wd)$ marginal cost is established.

7 Fast, Crash-Consistent Recovery

The pre-committer stable engine supplies a complementary recovery-cost baseline. A sealed segment loads an HNSW snapshot, whereas an unsealed legacy mutable segment replays its WAL through the graph builder. This is distinct from the new committer’s raw R/U recovery path. Table 5 shows that, from $N = 1\text{k}$ to 6k , snapshot load rises from 39 to 47 ms while rebuild rises from 196 to 1,484 ms; the gap grows from $5.0\times$ to $31.6\times$. Snapshot load is still linear in serialized size, with a smaller per-element constant, not constant time. A separate pre-committer plain-`fsync` run measured snapshot creation at 29.7/36.0/55.3 ms at the same sizes; because Table 5 uses `F_FULLFSYNC`, those creation numbers are context rather than a direct end-to-end comparison. The table is the seven-trial Apple result; the AWS cross-check reaches a $128\times$ gap at 6k (§3).

Extending this pre-committer sweep to $N = 20\text{k}$ gives 37 ms snapshot load versus 5.9 s graph rebuild, a $157\times$ gap. These timings justify periodic sealing for the legacy mutable-HNSW design; they should not be read as a measured recovery cost for the new raw-delta committer.

Table 6: Recall@10 vs. single-thread QPS on SIFT1M (Apple M4 Pro, $d=128$, $n=10^6$, $M=16$, $ef_{\text{construction}}=200$), ours vs. hnswlib on the identical dataset/params/host, measured against the dataset’s shipped exact ground truth (ID-set recall@10). QPS is single-run, and graph builds were not repeated.

ef	this work		hnswlib	
	recall@10	QPS	recall@10	QPS
10	0.770	22,190	0.709	20,829
32	0.931	10,114	0.904	9,907
64	0.977	5,851	0.963	5,758
100	0.990	4,045	0.983	3,903
200	0.997	2,240	0.996	2,200
500	0.999	1,023	0.999	1,040

8 Index Competitiveness

Table 6 is a sanity check that durability is not measured on a noncompetitive index. On SIFT1M, recall@10 rises from .770 to .999 as ef grows; against hnswlib on the same host and parameters, QPS is within a few percent and the measured recall points are slightly higher. Because QPS is single-run and graph builds are randomized, we claim only that this build is on the same practical recall–QPS frontier, not that it is deterministically superior.

9 Related Work

Dynamic and durable ANN. FreshDiskANN [3] and SPFresh [4] add streaming graph updates and describe log/snapshot recovery; LSM-VEC [5] distributes a proximity graph across LSM levels. P-HNSW [11] compares HNSW WAL replay and snapshot load on persistent memory. Our legacy measurements instead isolate synchronization cost on block storage, and our committer asks a different question: which acknowledged records may be discarded, and what top- k damage can that cause?

Turbopuffer is the closest operational composition [6]: successful writes are durably object-stored in a WAL, while committed but not yet indexed records remain searchable by an exhaustive tail, and a separate system monitors continuous recall [7]. That architecture motivates the same exact-tail pattern, but it does not use a recall budget to admit acknowledged *undurable* writes. VStream studies distributed streaming vector search and the freshness/recall tradeoff [8]; it does not define a storage-failure loss contract.

Bounds and measurement. TACT [17], PBS [18], and Pileus [19] bound consistency or staleness; Aether [16] and Silo [20] develop logging/group commit. ANN-Benchmarks [13], Big ANN [14], and VectorDBBench [15] omit crash recovery and synchronization modes. Our narrow contribution is to gate an opt-in weak ACK by strict or explicitly model-conditional top- k exposure, preserve that bound with an isolated exact tail, and check the resulting loss after production recovery.

10 Threats to Validity and Remaining Work

The legacy tax/recovery study uses seven trials on Apple and a separate AWS Linux cross-check; the real committer study uses one Apple M4 Pro and a small clustered synthetic workload. Thirty tail/write image executions expose substantial timing variance, but all share one base graph, query set, HNSW seed, and end-of-workload crash frontier. They are not independent graph or deployment-workload samples, and several performance ranges overlap. All pre-crash recall values are 1.0, so the loss experiment validates membership isolation rather than ANN-error interaction. There is no controlled query-cost sweep in W or d . The legacy sync-floor loop also omitted syscall return-value accounting, so we use it only to compare timing under requested sync modes, not to claim observed power survival.

Four implementation threats identified during review are now closed and tested. Latest scalar, metadata, and batch queries with $k < k_{\min}$ use stable visibility; every weak-visible batch query feeds the correlation guard; the oldest-tail age deadline preempts group delay; and failed `F_FULLFSYNC` returns an error without a plain-`fsync` fallback or stable ACK. The age deadline determines when fencing starts, not how long the device takes to complete synchronization. Injected-I/O tests cover fail-closed behavior, but not all real-device error and recovery modes.

Strict admission is distribution-free but permits only $\lfloor \varepsilon k_{\min} \rfloor$ weak records; for common small $k_{\min}$ and $\varepsilon < 1/k_{\min}$, it degenerates correctly to stable commit. Exchangeable mode can admit more, but hot records violate its mean model, as Table 4 demonstrates. The CUSUM limits future exposure after detection and cannot undo an already acknowledged tail. In the guard benchmark, image cloning waits for the alarm-triggered fence, so its zero-loss row does not test a crash during detection or synchronization. Total mutable search is $\Theta((|R| + W)d)$. The implementation and evaluation are insert-only, single-node, and unfiltered; updates/deletes retain stable semantics, and tenant filters, sharding, and replication remain open.

Finally, 15 `_exit` points test process death, 661 byte prefixes test the WAL parser, and cloned images test production reopen. None removes a device cache or reproduces arbitrary block persistence. The Linux `dm-log-writes` committer harness is implemented with an external ACK ledger, per-entry replay, normal journal recovery, and read-only verification, but its destructive physical run is pending suitable hardware. The older graph-rebuild recovery table is a pre-committer baseline, not a measurement of current raw-delta recovery.

11 Conclusion

ANN durability should be reported as a contract, not inferred from an API name. The two-host legacy study shows why: synchronization semantics change measured cost, stable group commit recovers much of that cost, and snapshot loading avoids graph-rebuilding replay. Those are established storage techniques measured on an ANN path, not new primitives.

The new result is an implemented opt-in weak-ACK path whose risk policy and query representation agree. A sole appender gates U , fenced WAL generations define recovery, and exact R/U layers keep volatile inserts out of HNSW topology. Across 15 process-crash frontiers, 661 WAL prefixes, and 2,880 recovered-query observations, stable records survive, lost IDs stay within U_{pre} , admission never overshoots, and the observed strict rows satisfy $\Delta^+ \leq L = M \leq \varepsilon$. Exchangeable mode is higher at the median in the canonical run, although its throughput range overlaps stable group commit, and it fails as a per-query guarantee on hot data. The guard responds by fencing and falling back to stable ACKs at substantial cost. The shipped default remains stable. General claims await larger workloads and an executed physical block-replay experiment.

Reproducibility

Run the committer artifact from the repository root:

```
make test
make committer-unit-test
make committer-crash-test
make committer-cut-test
make bench-recall-committer \
    RECALL_COMMIT_ARGS="--repetitions 30 \
    --output docs/paper/data"
make powerloss-committer
cd docs/paper
python3 plot_recall_committer.py
~/local/bin/tectonic -X compile honest-durability.tex
```

The portable targets produce 75 general unit tests, 24 policy/API tests, the 15/15 process-crash matrix, 661/661 byte-prefix sweep, and benchmark CSVs. On macOS, `powerloss-committer` runs the portable tests and reports the physical `dm-log-writes` branch unavailable; it is not a physical pass. That branch requires Linux, root, explicit destructive opt-in, and three distinct unmounted disposable devices as documented in `scripts/powerloss_committer.sh`.

The committer run used an Apple M4 Pro, macOS 26.5.2/Darwin 25.5.0, APFS on internal Apple NVMe, Apple clang 21.0.0, C++20 `-O2 -mcpu=native`, and `F_FULLFSYNC`. Benchmark defaults are 160 clustered base records, 25 writes, 32 concurrent queries, $d = 12$, $k = 10$, four writers, 30 repetitions, $ef = 32$, and seed 100. Aggregate, crash, raw-operation, stdout, environment, and SHA-256 files from the same code commit and run are tracked in `docs/paper/data`; no separately selected timing run is used. The legacy tax/group/recovery CSVs are archived pre-committer evidence: current `make bench-durability` exercises the raw-delta engine and is not their exact reproducer. Table 6 uses `make bench-ann` and `make bench-hnswlib` on `datasets/sift`; the data README records all SIFT1M SHA-256 values and pins hnswlib v0.9.0 commit `d9b3608c83d83b46c96e25088cb1d729b29dcfe9`.

References

- [1] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi:10.1109/TPAMI.2018.2889473. arXiv:1603.09320.
- [2] Yu. A. Malkov and D. A. Yashunin. hnswlib: A header-only C++/Python library for fast approximate nearest neighbor search. GitHub repository, `nmslib/hnswlib`, Apache-2.0, 2016–present. <https://github.com/nmslib/hnswlib>.
- [3] A. Singh, S. J. Subramanya, R. Krishnaswamy, and H. V. Simhadri. FreshDiskANN: A fast and accurate graph-based ANN index for streaming similarity search. arXiv preprint arXiv:2105.09613, 2021.
- [4] Y. Xu, H. Liang, J. Li, S. Xu, Q. Chen, Q. Zhang, C. Li, Z. Yang, F. Yang, Y. Yang, P. Cheng, and M. Yang. SPFresh: Incremental in-place update for billion-scale vector search. In *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 545–561, 2023. doi:10.1145/3600006.3613166.
- [5] S. Zhong, D. Mo, and S. Luo. LSM-VEC: A large-scale disk-based system for dynamic vector search. arXiv preprint arXiv:2505.17152, 2025.

- [6] Turbopuffer. Architecture: durable writes and unindexed-data search. Product documentation, accessed July 2026. <https://turbopuffer.com/docs/architecture>.
- [7] M. Gallant. Continuous recall measurement. Turbopuffer engineering blog, September 2024. <https://turbopuffer.com/blog/continuous-recall>.
- [8] S. Gong, H. Sun, Z. Fang, L. Liu, L. Chen, and Y. Gao. VStream: A distributed streaming vector search system. *Proc. VLDB Endowment (PVLDB)*, 18(6):1593–1606, 2025. doi:10.14778/3725688.3725692.
- [9] N. Upreti, H. V. Simhadri, H. S. Sundar, K. Sundaram, et al. Cost-effective, low-latency vector search with Azure Cosmos DB. *Proc. VLDB Endowment (PVLDB)*, 18(12):5166–5183, 2025. arXiv:2505.05885.
- [10] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou. The Faiss library. arXiv preprint arXiv:2401.08281, 2024.
- [11] H. Lee, T. Park, Y. Na, and W.-H. Kim. P-HNSW: Crash-consistent HNSW for vector databases on persistent memory. *Applied Sciences*, 15(19):10554, 2025. doi:10.3390/app151910554.
- [12] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu, K. Yu, Y. Yuan, Y. Zou, J. Long, Y. Cai, Z. Li, Z. Zhang, Y. Mo, J. Gu, R. Jiang, Y. Wei, and C. Xie. Milvus: A purpose-built vector data management system. In *Proc. 2021 ACM SIGMOD International Conference on Management of Data*, pages 2614–2627, 2021. doi:10.1145/3448016.3457550.
- [13] M. Aumüller, E. Bernhardsson, and A. Faithfull. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, 87:101374, 2020. doi:10.1016/j.is.2019.02.006.
- [14] H. V. Simhadri, M. Aumüller, A. Ingber, M. Douze, et al. Results of the Big ANN: NeurIPS’23 competition. arXiv preprint arXiv:2409.17424, 2024.
- [15] Zilliz. VectorDBBench: A benchmark tool for vector databases. Open-source software, GitHub, 2023. <https://github.com/zilliztech/VectorDBBench>.
- [16] R. Johnson, I. Pandis, R. Stoica, M. Athanassoulis, and A. Ailamaki. Aether: A scalable approach to logging. *Proc. VLDB Endowment (PVLDB)*, 3(1):681–692, 2010. doi:10.14778/1920841.1920928.
- [17] H. Yu and A. Vahdat. Design and evaluation of a conit-based continuous consistency model for replicated services. *ACM Trans. Computer Systems*, 20(3):239–282, 2002. doi:10.1145/566340.566342.
- [18] P. Bailis, S. Venkataraman, M. J. Franklin, J. M. Hellerstein, and I. Stoica. Probabilistically bounded staleness for practical partial quorums. *Proc. VLDB Endowment (PVLDB)*, 5(8):776–787, 2012. doi:10.14778/2212351.2212359.
- [19] D. B. Terry, V. Prabhakaran, R. Kotla, M. Balakrishnan, M. K. Aguilera, and H. Abu-Libdeh. Consistency-based service level agreements for cloud storage. In *Proc. 24th ACM Symposium on Operating Systems Principles (SOSP)*, pages 309–324, 2013. doi:10.1145/2517349.2522731.
- [20] S. Tu, W. Zheng, E. Kohler, B. Liskov, and S. Madden. Speedy transactions in multicore in-memory databases. In *Proc. 24th ACM Symposium on Operating Systems Principles (SOSP)*, pages 18–32, 2013. doi:10.1145/2517349.2522713.
- [21] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswani, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. All file systems are not created equal: On the complexity of crafting crash-consistent applications. In *Proc. 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 433–448, 2014.
- [22] M. Zheng, J. Tucek, D. Huang, F. Qin, M. Lillibridge, E. S. Yang, B. W. Zhao, and S. Singh. Torturing databases for fun and profit. In *Proc. 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 449–464, 2014.
- [23] J. Corbet. PostgreSQL’s fsync() surprise. LWN.net, 2018. <https://lwn.net/Articles/752063/>.